

# Ponteiros e Funções (cont.)

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz  
marcosleipnitz@ic.ufrj.br

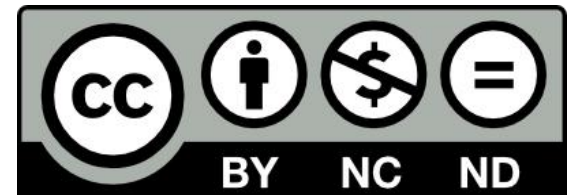


**UFRJ**

UNIVERSIDADE FEDERAL  
DO RIO DE JANEIRO



INSTITUTO DE  
COMPUTAÇÃO  
UFRJ



- Ponteiros
  - Operações com ponteiros
  - Ponteiros para ponteiros
  - Ponteiros para estruturas
- Funções
  - Passagem de arranjos como parâmetros
- Exemplos
- Exercícios

# Operações com Ponteiros

- Apenas duas operações aritméticas podem ser utilizadas com o endereço armazenado por um ponteiro: **adição** e **subtração**
- Podemos apenas somar e subtrair valores **INTEIROS**

```
p++; // soma +1 no endereço armazenado no ponteiro.  
p--; // subtrai 1 no endereço armazenado no ponteiro.  
p = p+15; // soma 15 no endereço armazenado no ponteiro.
```

- Operações **ILEGAIS** com ponteiros:
  - Dividir ou multiplicar ponteiros;
  - Somar o endereço de dois ponteiros;
  - **Não** se pode adicionar ou subtrair valores dos tipos **float** ou **double** de ponteiros.

# Operações com Ponteiros

- Sobre seu **conteúdo** apontado, valem todas as operações:

```
(*p)++; // incrementar o conteúdo da variável apontada  
pelo ponteiro p;
```

```
*p = (*p) * 10; //multiplica o conteúdo da variável  
apontada pelo ponteiro p por 10;
```

- Operações **relacionais**
  - == e != para saber se dois ponteiros são iguais ou diferentes.
  - >, <, >= e <= para saber qual ponteiro aponta para uma posição mais alta na memória.

# Operações com Ponteiros

```
#include <stdio.h>
int main(){
    int vet[] = {10, 20};
    int *p = vet;
    ++*p;
    printf("%d", *p);
    return 0;
}
```

→ **11**

```
#include <stdio.h>
int main(){
    int vet[] = {10, 20};
    int *p = vet;
    *p++;
    printf("%d", *p);
    return 0;
}
```

→ **20**

```
#include <stdio.h>
int main(){
    int vet[] = {10, 20};
    int *p = vet;
    *++p;
    printf("%d", *p);
    return 0;
}
```

→ **20**

**$++*p$  é equivalente a  $++(*p)$**

A precedência de  $*$  e  $++$  pré-fixado é a mesma, e associatividade é da direita para a esquerda

**$*p++$  é equivalente a  $*(p++)$**

A precedência do  $++$  pós-fixado é maior que a do  $*$ .

**$*++p$  é equivalente a  $*(++p)$**

A precedência de  $*$  e  $++$  pré-fixado é a mesma, e associatividade é da direita para a esquerda

# Operações com Ponteiros

- Ponteiros **genéricos**
  - Normalmente, um ponteiro aponta para um tipo **específico** de dado.
  - Um ponteiro genérico é um ponteiro que pode apontar para **qualquer** tipo de dado.
- Declaração:

```
void *<nome_ponteiro>
```
- Para acessar o **conteúdo** de um ponteiro genérico, antes é preciso convertê-lo para o tipo de ponteiro com o qual se deseja trabalhar através de **type cast**

```
void *p; // ponteiro genérico
int valor = 10;
p = &valor; // recebe o endereço de um inteiro
printf("Endereço em p: %p \n", p); // mostra o endereço
printf("Conteúdo apontado por p: %d \n", *(int*)p);
// mostra o conteúdo (interpretado como inteiro)
```

- A linguagem C permite criar ponteiros com diferentes **níveis** de apontamento.
  - É possível criar um ponteiro que aponte para outro ponteiro, criando assim vários níveis de apontamento.
  - Assim, um ponteiro poderá apontar para outro ponteiro, que, por sua vez, aponta para outro ponteiro, que aponta para um terceiro ponteiro e assim por diante.
- Podemos declarar um ponteiro para um ponteiro com a seguinte notação:

```
<tipo_ponteiro> **<nome_ponteiro>
```

- Acesso ao conteúdo
  - **\*\*nome\_ponteiro** é o conteúdo final da variável apontada;
  - **\*nome\_ponteiro** é o conteúdo do ponteiro intermediário.

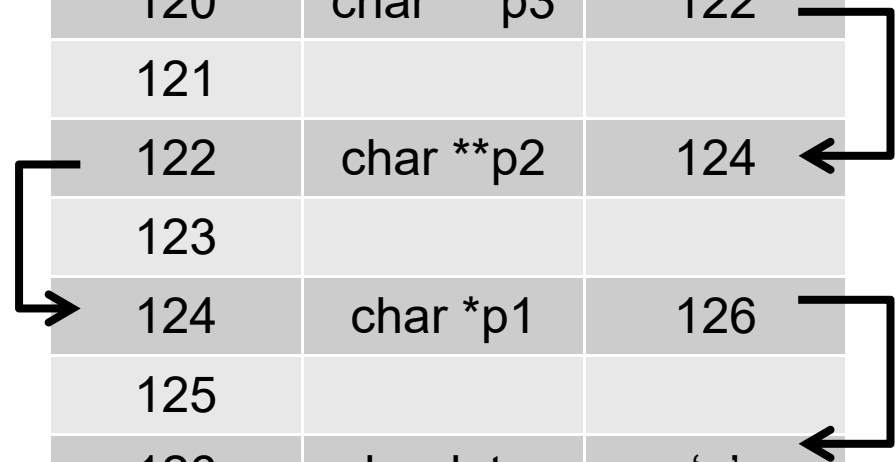
# Ponteiro para Ponteiro

- É a quantidade de **asteriscos (\*)** na declaração do ponteiro que indica o número de níveis de apontamento que ele possui.

```
char letra = 'a';  
char *p1;  
char **p2;  
char ***p3;
```

```
p1 = &letra;  
p2 = &p1;  
p3 = &p2;
```

| Memória  |            |          |
|----------|------------|----------|
| endereço | variável   | conteúdo |
| 119      |            |          |
| 120      | char ***p3 | 122      |
| 121      |            |          |
| 122      | char **p2  | 124      |
| 123      |            |          |
| 124      | char *p1   | 126      |
| 125      |            |          |
| 126      | char letra | 'a'      |
| 127      |            |          |



- Existem duas abordagens para acessar o **conteúdo** (atributos/campo) de um ponteiro para uma **estrutura**:
  - **Abordagem 1:** Devemos acessar o conteúdo do ponteiro para struct para somente depois acessar os seus campos e modificá-los.
    - Ex.: `(*ponteiro).nome_campo`
  - **Abordagem 2:** Podemos usar o operador seta (`->`)
    - Ex.: `ponteiro->nome_campo`
- Quando um ponteiro indica uma variável do tipo estrutura, para que se acesse os atributos desta variável, ao invés da combinação do operador “\*” com o operador “.” pode-se utilizar o operador “->”

# Ponteiros para Estruturas (Struct)

- Exemplo de uso do operador seta (->)

```
#include <stdio.h>
typedef struct
{
    char nome [30];
    int matricula;
} tAluno;
int main ()
{
    tAluno *x;
    tAluno y;
    x = &y;
    (*x).matricula = 2031;
    // x->matricula = 2031; //forma alternativa
    printf ("%d\n", y.matricula);
    return 0;
}
```

# Ponteiros para Estruturas (Struct)

- **Exercício:** Considere o tipo de dado **tEmpregado** definido abaixo para armazenar informações sobre um empregado. Implemente uma função para reajustar o salário de um dado empregado em 10%, caso ele tenha nascido antes de 1990, e de 7%, caso ele tenha nascido a partir de 1990.

```
typedef struct
{
    int dia, mes, ano;
} tData;

typedef struct
{
    char nome[40];
    tData dataNascimento, dataAdmissao;
    char sexo;
    float salario;
} tEmpregado;
```

# Ponteiros para Estruturas (Struct)

- Solução:

```
#include <stdio.h>
//Definição das estruturas dadas na ordem do exercício
void reajuste_salarial (tEmpregado *emp)
{
    if (emp->dataNascimento.ano<1990)
        emp->salario=emp->salario*1.1;
    else
        emp->salario=emp->salario*1.07;
}
```

- Forma alternativa:

```
void reajuste_salarial (tEmpregado *emp)
{
    if ( (*emp).dataNascimento.ano<1990)
        (*emp).salario= (*emp).salario*1.1;
    else
        (*emp).salario= (*emp).salario*1.07;
}
```

- Vetores **não podem** ser passados por **valor**.
  - Isso resultaria em cópia.
  - Copiar vetores muitos grandes (milhões de posições) é dispendioso, tanto em termos de tempo de processamento quanto de memória.
- Vetores são **sempre** passados por **referência**.
  - Modificações realizadas no vetor, dentro da função que o recebeu como parâmetro, são refletidas no vetor passado como parâmetro pela função que realizou a chamada.
- Apenas o **endereço do primeiro elemento** do vetor é passado (**v**, ou seja, **&v[0]**)
  - A função que recebeu o vetor não tem como saber qual é o **tamanho** dele sem informação adicional.
  - Por isso, para construir **funções genéricas**, capazes de lidar com qualquer vetor (de qualquer tamanho) passado como parâmetro, também precisamos passar o **tamanho** do vetor como parâmetro para a função.

# Arranjos como Parâmetros de Funções

- Declaração de um parâmetro formal **vetor** em uma **função/procedimento**.

`<tipo> <nome>[]` ou `<tipo> *<nome>`

- `<tipo>`: corresponde ao tipo dos elementos do vetor.
- `<nome>`: é o nome atribuído ao vetor.
- `[]`: indica que a variável é do tipo vetor.
  - Pode ser utilizado sem um valor, pois em C não interessa qual a dimensão do vetor que é passado a uma função, mas sim o tipo dos seus elementos.
- `*`: indica que é um ponteiro para o tipo primitivo de dado do vetor.

# Arranjos como Parâmetros de Funções

- Como especificar uma função que recebe um vetor **v** de inteiros e um valor **n** como parâmetros e preenche o vetor com esse valor?

```
#include<stdio.h>
#define QTD1 5
#define QTD2 10
void inicializa(int *v,int n, int tamanho){
    for(int i=0;i<tamanho;i++){
        *(v+i) = n;
    }
}
void imprime(int *v, int tamanho){
    for(int i=0;i<tamanho;i++){
        printf("%d\t",*(v+i));
    }
    printf("\n");
}
int main(){
    int v1[QTD1],v2[QTD2];
    inicializa(v1,1,QTD1);
    inicializa(v2,2,QTD2);
    imprime(v1,QTD1);
    imprime(v2,QTD2);
    return 0;
}
```

Parâmetro necessário para permitir reuso da função com vetores de diferentes tamanhos.

Função inicializa sendo chamada com vetores de tamanhos diferentes.

Função imprime sendo chamada com vetores de tamanhos diferentes.

# Arranjos como Parâmetros de Funções

- Como especificar uma função que recebe um vetor **v** de inteiros e um valor **n** como parâmetros e preenche o vetor com esse valor?

```
#include<stdio.h>
#define QTD1 5
#define QTD2 10
void inicializa(int *v, int n, int tamanho){
    for(int i=0; i<tamanho; i++){
        *(v+i) = n;
    }
}
void imprime(int *v, int tamanho){
    for(int i=0; i<tamanho; i++){
        printf("%d\t", *(v+i));
    }
    printf("\n");
}
int main(){
    int v1[QTD1], v2[QTD2];
    inicializa(v1, 1, QTD1);
    inicializa(v2, 2, QTD2);
    imprime(v1, QTD1);
    imprime(v2, QTD2);
    return 0;
}
```

Diagram illustrating the equivalence of pointer notation and array notation in the code:

- The expression `*(v+i)` in the `inicializa` function is equivalent to `v[i]`.
- The expression `*(v+i)` in the `imprime` function is equivalent to `v[i]`.

- Como passamos **strings** como parâmetros de funções?
  - Strings são vetores de caracteres que contém um caractere **terminador** (`'\0'`).
  - A passagem de strings por parâmetro segue os mesmos princípios da passagem de vetores por parâmetro!
  - Em geral, no caso de manipulação de strings, não é necessário passar o seu tamanho como parâmetro adicional.
- **Exemplo:** Escreva uma função que recebe uma string como parâmetro, que pode representar mais de uma frase, e que modifica a string para que ela contenha apenas a primeira frase (o restante do conteúdo é descartado). Assuma que uma frase sempre termina com `"."` (ponto final), `"!"` (ponto de exclamação) ou `"?"` (ponto de interrogação).

# Arranjos como Parâmetros de Funções

```
#include <stdio.h>
#include <string.h>
void primeiraFrase(char *texto){
    int i=0;
    while(texto[i] != '\0'){
        if(texto[i] == '.' || texto[i] == '!' || texto[i] == '?'){
            texto[i+1] = '\0';
        }
        i++;
    }
}

int main() {
    char string[41]; // Tamanho máximo de 40 + 1 para incluir o caractere '\0'
    printf("Digite um texto (máximo 40 caracteres): ");
    fgets(string, sizeof(string), stdin); // Lê até sizeof(string) - 1 caracteres
    // ou até encontrar '\n'. Inclui o '\n' (se couber) e o '\0' (sempre) no final.
    string[strcspn(string, "\n")] = '\0'; // remove o '\n' se foi incluído
    printf("Você digitou: %s\n", string);
    primeiraFrase(string); // Descarta informação desnecessária
    printf("Primeira frase: %s\n", string);
    return 0;
}
```

- A linguagem C não provê operadores para processar strings diretamente (a biblioteca **string.h** oferece um conjunto de funções para manipulação de strings)
- Tomaremos como exemplo a função **strcpy(dest, fonte)**: copia caracter a caracter a string **fonte** para a string **dest**.
- Veremos diferentes possibilidades de implementação dessa função.

# Arranjos como Parâmetros de Funções

- Implementação de **strcpy** usando vetores:

```
#include <stdio.h>

void strcpy1 (char *dest, char *fonte)
{
    int i = 0;
    while ((dest[i] = fonte[i]) != '\0')
        i++;
}

int main()
{
    char a[100], b[] = "Ola mundo";
    strcpy1 (a, b);
    printf("a: %s\nb: %s\n", a, b);
    return 0;
}
```

```
a: Ola mundo
b: Ola mundo
```

# Arranjos como Parâmetros de Funções

- Implementação de **strcpy** usando ponteiros:

```
void strcpy2 (char *dest, char *fonte)
{
    while ((*dest = *fonte) != '\0')
    {
        dest++;
        fonte++;
    }
}
```

```
void strcpy3 (char *dest, char *fonte)
{
    while ((*dest++ = *fonte++) != '\0');
}
```

```
void strcpy4 (char *dest, char *fonte)
{
    while (*dest++ = *fonte++);
}
```

# Arranjos como Parâmetros de Funções

- Passagem de **matrizes** como parâmetro
  - A passagem de arranjos multidimensionais para uma função é realizada indicando no cabeçalho desta, **obrigatoriamente**, o número de elementos de cada um das **n-1** dimensões à direita.
  - Apenas o tamanho da dimensão mais à esquerda pode ser omitida, colocando-se [ ].

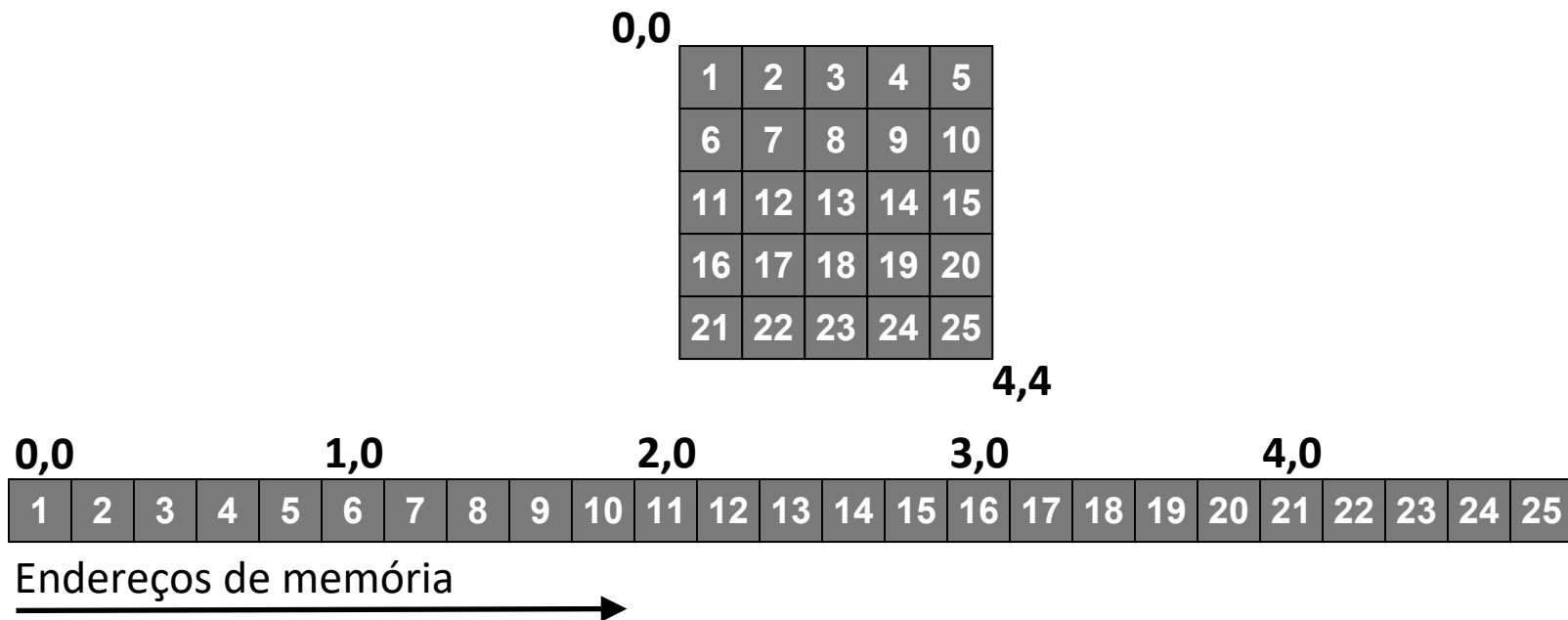
```
void funcao1 (float array1[][10]);  
void funcao2 (int array2[][20][30]);  
/*a primeira dimensão não precisa ser especificada, mas as  
   demais DEVEM!*/
```

- **Exemplo:** Construir uma função **inicM()** que inicializa uma matriz quadrada com 9 elementos e outra função **mostraM()** que mostra os elementos dessa matriz.

# Arranjos como Parâmetros de Funções

- Apesar de matrizes terem mais de uma **dimensão**, na memória os dados são armazenados **linearmente**.

Ex.: `int matriz[5][5]`.



- Exemplo:** Construir uma função `inicM()` que inicializa uma matriz quadrada com 9 elementos e outra função `mostraM()` que mostra os elementos dessa matriz.

# Arranjos como Parâmetros de Funções

```
#include <stdio.h>
#define DIM 3
void inicM(int s[ ][DIM])
{
    int i, j;
    for (i=0; i<DIM; i++)
        for (j=0; j <DIM; j++)
            s[i][j] = 0;
}
void mostraM (int s[DIM][DIM])
{
    int i, j;
    for (i=0; i<DIM; i++)
    {
        for (j=0; j<DIM; j++)
            printf(" [%d] ", s[i][j]);
        printf("\n");
    }
}
int main()
{
    int s[DIM][DIM];
    inicM(s);
    mostraM (s);
    return 0;
}
```

Vetores para ponteiros:

`* (s[i]+j)`

Ponteiros para ponteiros:

`* (* (s+i)+j)`

Saída:

|     |     |     |
|-----|-----|-----|
| [0] | [0] | [0] |
| [0] | [0] | [0] |
| [0] | [0] | [0] |